

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA: AZZAMY GUSALIM HERFINO

NIM: 109082500091

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Pengurutan (Sorting)

Pengurutan (sorting) adalah suatu proses menyusun kembali sekumpulan objek atau data ke dalam suatu urutan tertentu berdasarkan aturan tertentu. Pengurutan umumnya dilakukan terhadap nilai kunci (key) dari masing-masing data. Secara umum, terdapat dua jenis pengurutan, yaitu:

1. **Membesar (Ascending)**: Data diurutkan dari nilai yang paling kecil hingga nilai yang paling besar.
2. **Mengecil (Descending)**: Data diurutkan dari nilai yang paling besar hingga nilai yang paling kecil.

Dalam algoritma dan pemrograman, terdapat berbagai macam metode pengurutan, dua di antaranya yang umum digunakan untuk mengurutkan kumpulan data pada array adalah Selection Sort dan Insertion Sort. Keduanya memiliki kompleksitas waktu $O(n^2)$ pada kasus rata-rata dan terburuk, di mana n adalah jumlah data.

2. Selection Sort (Pengurutan secara Seleksi)

Selection Sort adalah algoritma pengurutan yang beroperasi dengan ide dasar mencari nilai ekstrim (nilai terkecil untuk ascending atau terbesar untuk descending) pada sekumpulan data yang belum terurut, kemudian meletakkan nilai tersebut pada posisi yang seharusnya.

Algoritma ini melibatkan dua proses utama:

1. **Pencarian indeks nilai ekstrim**: Melakukan iterasi pada sisa rentang data untuk menemukan indeks dengan nilai paling minimum atau maksimum.
2. **Pertukaran nilai (Swap)**: Menukar posisi elemen di indeks ekstrim tersebut dengan elemen di posisi paling kiri pada rentang data yang sedang dievaluasi.

Langkah-langkah algoritma Selection Sort (contoh untuk ascending):

1. Cari nilai terkecil di dalam rentang data yang tersisa (belum terurut).
2. Pindahkan atau tukar tempat nilai tersebut dengan data yang berada pada posisi paling kiri pada rentang data tersisa tersebut.
3. Ulangi proses pertama dan kedua ini dengan menggeser batas awal data yang belum terurut ke kanan sejauh satu langkah, sampai hanya tersisa satu data saja.

2.3 Insertion Sort (Pengurutan secara Penyisipan)

Insertion Sort adalah algoritma pengurutan yang bekerja dengan cara mengambil elemen data satu per satu, kemudian menyisipkan elemen tersebut ke posisi yang tepat pada bagian array yang sudah diurutkan.

Berbeda dengan Selection Sort, algoritma ini tidak melakukan pencarian nilai ekstrim terlebih dahulu di seluruh array sisa. Insertion Sort hanya memilih suatu nilai tertentu (yang sedang dievaluasi), kemudian mencari posisinya secara pencarian sekuensial (sequential search) di dalam sub-kumpulan data yang sudah terurut di sebelah kirinya. Algoritma ini melibatkan proses pencarian posisi dan proses penggeseran data (shifting).

Langkah-langkah algoritma Insertion Sort (contoh untuk descending):

1. Asumsikan elemen pertama sudah berada pada posisi yang terurut. Ambil elemen kedua sebagai data yang akan disisipkan.
2. Untuk data yang belum terurut tersebut, bandingkan nilainya dengan kumpulan data yang sudah diurutkan sebelumnya (berada di sebelah kirinya).
3. Geser data yang sudah terurut tersebut (ke kanan) satu per satu jika posisinya belum sesuai, sehingga ada satu ruang kosong tercipta.
4. Masukkan data yang belum terurut ke dalam ruang kosong tersebut dengan tetap menjaga keterurutan.
5. Ulangi proses iterasi ini (langkah 1-4) untuk seluruh sisa data yang belum terurut.

B. Guided

1. Sorting

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(arr); j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        if idx_min != i {
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

Output :

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Guided\Guided-1> go run no1.go
Masukkan data angka indeks ke-0 : 5
Masukkan data angka indeks ke-1 : 3
Masukkan data angka indeks ke-2 : 2
Masukkan data angka indeks ke-3 : 4
Masukkan data angka indeks ke-4 : 1
Masukkan data angka indeks ke-5 : 6
Masukkan data angka indeks ke-6 : 8
Masukkan data angka indeks ke-7 : 7

=== SEBELUM SORTING ===
5 3 2 4 1 6 8 7
=== SETELAH SORTING ===
1 2 3 4 5 6 7 8
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Guided\Guided-1> 
```

2. Sorting

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK       float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK {
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}
```

```

    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Printf(" === SEBELUM SORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SRORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

OUTPUT DEBUG CONSOLE TERMINAL PORTS PROBLEMS

```
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusali-Herfino\WEEK 12\Guided\Guided-2> go run no2.go
--- Masukkan Data Mahasiswa ke-0 ---
Nama : zemy
NIM : 123
IPK : 3.8
--- Masukkan Data Mahasiswa ke-1 ---
Nama : azam
NIM : 345
IPK : 3.77
--- Masukkan Data Mahasiswa ke-2 ---
Nama : salim
NIM : 567
IPK : 3.9
--- Masukkan Data Mahasiswa ke-3 ---
Nama : herfino
NIM : 789
IPK : 3.95
--- Masukkan Data Mahasiswa ke-4 ---
Nama : azzamy
NIM : 321
IPK : 3.85

=== SEBELUM SORTING === --- Data Mahasiswa ke-0 ---
Nama : zemy
NIM : 123
IPK : 3.80
--- Data Mahasiswa ke-1 ---
Nama : azam
NIM : 345
IPK : 3.77
--- Data Mahasiswa ke-2 ---
Nama : salim
NIM : 567
IPK : 3.90
--- Data Mahasiswa ke-3 ---
Nama : herfino
NIM : 789
IPK : 3.95
--- Data Mahasiswa ke-4 ---
Nama : azzamy
NIM : 321
IPK : 3.85
=====
```

```
=== SETELAH SORTING ===
--- Data Mahasiswa ke-0 ---
Nama : azam
NIM : 345
IPK : 3.77
--- Data Mahasiswa ke-1 ---
Nama : zemy
NIM : 123
IPK : 3.80
--- Data Mahasiswa ke-2 ---
Nama : azzamy
NIM : 321
IPK : 3.85
--- Data Mahasiswa ke-3 ---
Nama : salim
NIM : 567
IPK : 3.90
--- Data Mahasiswa ke-4 ---
Nama : herfino
NIM : 789
IPK : 3.95
=====
```

```
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusali-Herfino\WEEK 12\Guided\Guided-2> █
```

3. Sorting

```
package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending): ")
}
```



```

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```

Output :

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Guided\Guided-3> go run no3.go
Masukkan jumlah data: 4
Masukkan data:
5 2 6 3

Data sebelum sorting:
5 2 6 3

Data setelah sorting (Descending):
6 5 3 2
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Guided\Guided-3> 

```

4. Sorting

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    ipk  float64
}

type arrMHS [100]mahasiswa

func insertionSortStruct(T *arrMHS, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
    }
}

```

```

        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
        T[j+1] = temp
    }
}

func main() {
    var data arrMHS
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i+1)

        fmt.Print("Nama : ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM : ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK : ")
        fmt.Scan(&data[i].ipk)
    }

    fmt.Printf("\nData sebelum sorting:\n")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData setelah sorting (Descending berdasarkan
nama): ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }
}

```

Output :

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Guided\Guided-4> go run no4.go
Masukkan jumlah data: 3

Data mahasiswa ke- 1
Nama : zemmyy
NIM : 123
IPK : 3.8

Data mahasiswa ke- 2
Nama : salim
NIM : 456
IPK : 3.95

Data mahasiswa ke- 3
Nama : herfino
NIM : 789
IPK : 3.7

Data sebelum sorting:
zemmyy 123 3.8
salim 456 3.95
herfino 789 3.7

Data setelah sorting (Descending berdasarkan nama):
zemmyy 123 3.8
salim 456 3.95
herfino 789 3.7
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Guided\Guided-4> 
```

C. Unguided

1. Rumah Kerabat

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i+1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
    }
}
```

```

        temp := arr[idxMin]
        arr[idxMin] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var rumah []int
        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            rumah = append(rumah, nomor)
        }

        selectionSort(rumah, m)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[j])
        }
        fmt.Println()
    }
}

```

Output :

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no1> go run no1.go
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no1> 

```

Penjelasan :

Kode tersebut berfungsi untuk mengurutkan beberapa kelompok nomor rumah dari yang terkecil ke terbesar menggunakan algoritma selection sort. Program memulainya dengan membaca angka n untuk mengetahui jumlah daerah yang akan diproses, lalu menjalankan perulangan sebanyak n kali. Pada setiap putarannya, program membaca jumlah rumah m, menangkap setiap nomor rumah satu per satu dari input pengguna, dan menyimpannya ke dalam sebuah slice kosong bernama rumah menggunakan perintah append. Setelah satu kelompok data terkumpul, fungsi selectionSort akan dipanggil untuk menyusun data tersebut dengan cara mencari angka terkecil pada porsi data yang belum terurut lalu menukarnya ke posisi paling depan secara terus-menerus. Pada akhirnya, program mencetak daftar nomor rumah yang sudah tersusun rapi dengan pemisah spasi, dan siklus ini diulang terus sampai semua daerah selesai diproses.

2. Kerabat Dekat

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
```

```

        idxMin := i
        for j := i+1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        temp := arr[idxMin]
        arr[idxMin] = arr[i]
        arr[i] = temp
    }
}

func selectionSortDesc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i+1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        temp := arr[idxMax]
        arr[idxMax] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }
    }
}

```



```

    }

    selectionSortAsc(ganjil, len(ganjil))
    selectionSortDesc(genap, len(genap))

    first := true
    for j := 0; j < len(ganjil); j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(ganjil[j])
        first = false
    }

    for j := 0; j < len(genap); j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(genap[j])
        first = false
    }
    fmt.Println()
}
}

```

Output :

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no2> go run no2.go
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no2> 

```

Penjelasan :

Kode tersebut bertugas untuk memisahkan dan mengurutkan nomor rumah ganjil dan genap dengan aturan yang berbeda dalam beberapa kelompok data. Awalnya, program membaca angka 3 di terminal sebagai penanda bahwa ada tiga daerah yang akan diproses. Pada setiap daerahnya, program membaca

jumlah rumah terlebih dahulu (misalnya angka 5 pada baris kedua), lalu membaca sederet nomor rumah tersebut satu per satu dan langsung memisahkannya ke dalam slice ganjil atau genap menggunakan perintah `append`. Setelah datanya terpisah, kelompok angka ganjil diurutkan dari yang terkecil ke terbesar (*ascending*) menggunakan fungsi `selectionSortAsc`, sedangkan kelompok angka genap diurutkan dari yang terbesar ke terkecil (*descending*) menggunakan fungsi `selectionSortDesc`. Seperti yang terlihat pada contoh eksekusimu, deret input 2 1 7 9 13 diproses sehingga angka ganjilnya tersusun naik menjadi 1 7 9 13 lalu langsung disambung dengan sisa angka genapnya yaitu 2, sehingga memunculkan output 1 7 9 13 2 dalam satu baris, dan siklus pemisahan serta pengurutan ini akan terus diulang sampai seluruh data daerah selesai dieksekusi.

3. Jarak Data

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main

import "fmt"
```

```

func insertionSort(arr []int, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j = j - 1
        }
        arr[j] = temp
    }
}

func main() {
    var arr []int
    var input int

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        arr = append(arr, input)
    }

    n := len(arr)
    if n > 0 {
        insertionSort(arr, n)
    }

    for i := 0; i < n; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(arr[i])
    }
    fmt.Println()

    if n < 2 {
        fmt.Println("Data berjarak tidak tetap")
    } else {
        jarak := arr[1] - arr[0]
        tetap := true
        for i := 1; i < n-1; i++ {
            if arr[i+1]-arr[i] != jarak {
                tetap = false
            }
        }
    }
}

```

```

        break
    }
}

if tetap {
    fmt.Printf("Data berjarak %d\n", jarak)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}
}

```

Output :

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no3> go run no3.go
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no3> 

```

Penjelasan :

Kode tersebut berfungsi untuk mengurutkan sekumpulan angka sekaligus mengecek apakah selisih (jarak) antar angkanya selalu sama atau beraturan. Program memulainya dengan membaca input secara terus-menerus dan menyimpannya ke dalam sebuah slice sampai menemukan angka negatif (seperti -31 pada contoh terminal) yang bertugas sebagai penanda untuk berhenti membaca. Setelah kumpulan angka positif tersebut ditampung, program memanggil fungsi `insertionSort` untuk menyusunnya dari nilai terkecil ke terbesar dengan cara menggeser dan menyisipkan tiap angka ke posisi yang seharusnya. Selanjutnya, program mencetak daftar angka yang sudah terurut tersebut (sehingga susunannya menjadi 1 2 4 8 14 26 32 38 40), lalu mulai mengecek selisih antara setiap angka yang berdekatan. Karena jarak antar angka pada hasil urutan tersebut tidak konsisten—contohnya jarak dari 1 ke 2 adalah 1, sedangkan dari 2 ke 4 adalah 2—maka program menyimpulkan bahwa polanya tidak beraturan dan mencetak pesan "Data berjarak tidak tetap" sebagai output akhirnya.

4. XX

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
 terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
 Catatan: Gunakan metoda Insertion sort}

procedure CetakSTerbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
 eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
 dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
 rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}
```

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul, &pustaka[i].penulis,
            &pustaka[i].penerbit, &pustaka[i].eksemplar,
            &pustaka[i].tahun, &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
    maxRating := pustaka[0].rating
    idx := 0
    for i := 1; i < n; i++ {
        if pustaka[i].rating > maxRating {
            maxRating = pustaka[i].rating
            idx = i
        }
    }
    fmt.Printf("%s %s %s %d\n", pustaka[idx].judul, pustaka[idx].penulis,
        pustaka[idx].penerbit, pustaka[idx].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i
        for j > 0 && temp.rating > pustaka[j-1].rating {
```

```

        pustaka[j] = pustaka[j-1]
        j--
    }
    pustaka[j] = temp
}
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }
    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    kiri := 0
    kanan := n - 1
    found := false
    idx := -1

    for kiri <= kanan && !found {
        tengah := (kiri + kanan) / 2
        if pustaka[tengah].rating == r {
            found = true
            idx = tengah
        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if found {
        fmt.Printf("%s %s %s %d %d %d\n", pustaka[idx].judul,
pustaka[idx].penulis, pustaka[idx].penerbit, pustaka[idx].tahun,
pustaka[idx].eksemplar, pustaka[idx].rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

func main() {

```

```

var pustaka DaftarBuku
var n, r int

DaftarkanBuku(&pustaka, &n)
CetakTerfavorit(pustaka, n)
UrutBuku(&pustaka, n)
Cetak5Terbaru(pustaka, n)

fmt.Scan(&r)
CariBuku(pustaka, n, r)
}

```

Output :

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PROBLEMS

PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no4> go run no4.go
6
B001 Hai1_Mary Andy_Weir Random_House 15 2021 95
B002 Go_ in Action William Kennedy Manning 10 2015 88
B003 AT Modern Approach Stuart Russell Pearson 20 2020 98
B004 Deep_ Learning Ian Goodfellow MIT Press 25 2016 92
Hai1_Mary Andy_Weir Random_House 2021
Hai1_Mary
Go_
Manning
AT
Russell
Manning 10 2015 0 88 0
PS C:\Users\useru\OneDrive\azzamy\praktikumsem2\1090582500091_Azzamy-Gusalim-Herfino\WEEK 12\Unguided\Unguided-no4> 

```

Penjelasan :

Kode yang ada di Canvas tersebut merupakan program manajemen perpustakaan sederhana untuk mendata dan memproses daftar buku beserta ratingnya. Awalnya, program membaca angka 6 sebagai jumlah buku, lalu menampung semua detail inputan keenam buku tersebut (seperti ID, judul, penulis, dan rating) ke dalam sebuah array. Setelah data tersimpan, program memanggil fungsi CetakTerfavorit untuk langsung menelusuri dan mencetak detail buku dengan rating tertinggi dari daftar mentah tersebut, yang pada contoh ini memunculkan buku "Pemrograman_Go" (rating 95). Kemudian, program menggunakan fungsi UrutBuku untuk menyusun ulang seluruh daftar buku dari rating yang paling besar ke yang paling kecil menggunakan teknik insertion sort. Begitu datanya sudah berurutan secara menurun, program memanggil fungsi Cetak5Terbaru untuk sekadar menampilkan lima judul buku teratas secara berbaris ke bawah. Pada tahap terakhir, program membaca input angka 88 di bagian

paling bawah terminal, lalu fungsi CariBuku melakukan pencarian efisien menggunakan metode binary search pada daftar yang sudah terurut tadi untuk menemukan dan mencetak rincian lengkap dari buku "Basis_Data" yang secara spesifik memiliki rating 88 tersebut.

D. Kesimpulan

Kesimpulan dari pembelajaran modul ini adalah bahwa algoritma pengurutan seperti Selection Sort dan Insertion Sort memiliki peran yang sangat esensial dalam menyusun dan mengelola sekumpulan data agar menjadi lebih rapi dan terstruktur. Melalui penerapan kedua metode pengurutan tersebut yang dikombinasikan dengan teknik Binary Search serta pemanfaatan struktur data seperti slice dan struct, berbagai variasi masalah pemrograman—mulai dari pengurutan angka ganjil-genap berdasar kondisi, pengecekan jarak antar elemen, hingga manajemen data pada sistem perpustakaan—dapat diselesaikan secara lebih efisien dan terarah. Secara keseluruhan, pemahaman yang kuat terhadap logika perbandingan, pergeseran, dan penukaran posisi elemen menjadi landasan utama yang sangat dibutuhkan untuk merancang alur program yang akurat, sistematis, dan mampu menangani berbagai bentuk manipulasi data.